

Universidade Federal de Goiás — Instituto de Informática
INF/UFG 2026-1 · Algoritmos e Estruturas de Dados 2
Prof. André L. Moura

Documentação Técnica do Projeto Final

MinRoute

Sistema de Navegação com Algoritmo de Dijkstra

Equipe de Trabalho

Leonardo Ferreira Amichi
Lucas Garcia de Lacerda

Goiânia — Goiás, 2026

Contents

1	Introdução	2
1.1	Visão Geral do Sistema	2
1.2	Escopo	2
1.3	Objetivos do Documento	2
2	Descrição Geral do Sistema	2
2.1	Contexto e Motivação	2
2.2	Arquitetura Híbrida	3
2.3	Estrutura de Diretórios	3
3	Requisitos do Sistema	4
3.1	Requisitos Funcionais	4
3.2	Requisitos Não Funcionais	4
4	Modelagem UML	5
4.1	Diagrama de Casos de Uso	5
4.2	Diagrama de Classes	5
4.3	Diagrama de Sequência — Calcular Menor Caminho	6
5	Estruturas de Dados	7
5.1	Lista de Adjacência via Array de Vértices	7
5.2	Fila de Prioridade — Heap Mínima	8
5.3	Vetores de Rastreamento	8
5.4	Fluxo do Algoritmo de Dijkstra	8
6	Arquitetura da Interface Gráfica	9
6.1	Componentes PyQt6	9
6.2	Otimização de Renderização	10
6.3	Interface de Comunicação IPC (JSON)	10
7	Tratamento de Exceções e Casos Especiais	10
8	Guia de Instalação e Execução	11
8.1	Pré-Requisitos	11
8.2	Instalação das Dependências Python	11
8.3	Compilação do Módulo Java	11
8.4	Execução	11
8.5	Instalador Autônomo	12
9	Referências	12

1. Introdução

1.1 Visão Geral do Sistema

O **MinRoute** é um sistema de navegação primitivo desenvolvido como projeto final da disciplina de Algoritmos e Estruturas de Dados 2 (AED2). O sistema permite ao usuário importar malhas viárias reais, selecionar visualmente dois pontos de interesse — origem (s) e destino (t) — e computar graficamente o caminho de menor custo entre eles, com suporte a vias de mão única e mão dupla.

O motor computacional é apoiado exclusivamente no **Algoritmo de Dijkstra**, implementado sobre estruturas de dados eficientes para garantir o processamento de grafos com milhares de vértices dentro dos limites de tempo especificados.

1.2 Escopo

O sistema abrange as seguintes capacidades:

- Importação e conversão de arquivos de mapa nos formatos `.txt`, `.poly` e `.osm` (XML georreferenciado)
- Renderização interativa de grafos com navegação (pan e zoom)
- Cálculo e exibição visual do menor caminho com estatísticas de execução
- Edição dinâmica do grafo (inserção e remoção de vértices e arestas)

O sistema **não** contempla: roteamento em tempo real com GPS, atualização dinâmica de tráfego, ou suporte a mais de dois pontos de rota simultâneos.

1.3 Objetivos do Documento

Este documento serve como especificação formal do sistema MinRoute, cobrindo requisitos funcionais e não funcionais, arquitetura, modelagem UML, estruturas de dados, fluxo algorítmico e guia de execução. Destina-se à equipe de desenvolvimento, ao professor avaliador e a futuros mantenedores do código.

2. Descrição Geral do Sistema

2.1 Contexto e Motivação

Sistemas de navegação são aplicações clássicas de grafos ponderados. A escolha do Algoritmo de Dijkstra justifica-se por sua garantia de otimalidade em grafos com pesos não negativos — condição compatível com distâncias físicas entre intersecções viárias — e pela maturidade de sua implementação com Heaps Mínimas, que garante complexidade $O(E \log V)$.

Grafos de malhas urbanas reais são tipicamente esparsos: cada intersecção conecta-se a poucos vizinhos (em média 2 a 4), tornando a representação por Lista de Adjacência muito mais eficiente que a Matriz de Adjacência ($O(V^2)$).

2.2 Arquitetura Híbrida

O sistema adota uma arquitetura híbrida em dois processos, separando responsabilidades de forma clara:

Back-end (Java)

Motor algorítmico de alta performance. Responsável pela leitura dos arquivos de mapa, construção da Lista de Adjacência, execução do Dijkstra com Heap Mínima e serialização dos resultados em JSON via STDOUT.

Front-end (Python/PyQt6)

Interface gráfica responsiva. Responsável pela renderização do mapa via `QGraphicsScene`, captura de eventos de mouse, invocação do back-end Java como subprocesso, e exibição dos resultados.

A comunicação entre os processos (IPC) é feita via `subprocess.run()` com passagem de argumentos e leitura de STDOUT em JSON, eliminando dependências de rede ou sockets.

2.3 Estrutura de Diretórios

```

1 MinRoute/
2 |-- Documentacao/
3 |   '-- MinRoute_Documentacao.pdf
4 |-- data/
5 |   '-- mapa_editado.poly      (snapshot do grafo para IPC)
6 |-- src/
7 |   |-- core/                  (motor matematico - Java)
8 |   |   |-- DijkstraCore.java (algoritmo de Dijkstra + fila de
9 |   |   |   |-- Grafo.java      (array de vertices + insercao de
10 |   |   |   |-- ParserOSM.java  (parser XML/OSM georreferenciado)
11 |   |   |   '-- ParserPoly.java (parser de malhas cartesianas .
12 |   |   |       poly)
13 |   |-- ui/                    (interface grafica - Python/PyQt6)
14 |   |   |-- app.py              (janela principal + controle de
15 |   |   |   |-- icons.py        (fabrica de icones QIcon)
16 |   |   |   '-- renderer.py     (QGraphicsScene + renderizacao de
17 |   |   |       nos/arestas)
18 |   '-- utils/
19 |       |-- java_bridge.py      (invocador de subprocesso + parse
20 |       |   |-- map_model.py    (calculo de nos proximos + edicao
21 |       |       |-- README.md

```

Listing 1: Estrutura de diretórios do projeto MinRoute

3. Requisitos do Sistema

3.1 Requisitos Funcionais

A Tabela 1 lista os requisitos funcionais do sistema com sua descrição, pontuação e status de implementação planejada.

Table 1: Requisitos Funcionais do MinRoute

ID	Descrição	Pontos	Módulo
RF01	Importar mapas reais de arquivos <code>.txt</code> , <code>.poly</code> ou <code>.osm</code> e converter para grafos	0,75	ParserOSM, ParserPoly
RF02	Enumerar vértices e rotular arestas com pesos (distâncias)	0,20	renderer.py
RF03	Selecionar/desfazer origem e destino com marcas em cores distintas	0,20	app.py
RF04	Calcular e exibir o menor caminho em cor diferenciada	1,50	DijkstraCore, renderer.py
RF05	Criar e editar o grafo (adicionar/remover vértices e arestas) com mouse	1,10	map_model.py, app.py
RF06	Suporte a grafos não direcionados e direcionados (mão dupla/única)	1,30	Grafo.java
RF07	Exibir estatísticas: tempo de processamento, nós explorados, custo total	0,75	DijkstraCore, app.py
RF08	Copiar imagem do grafo para a área de transferência	0,20	app.py
Total		6,00	

3.2 Requisitos Não Funcionais

Table 2: Requisitos Não Funcionais do MinRoute

ID	Descrição	Pontos	Categoria
RNF01	Vértices e arestas do grafo com cores distintas	0,25	Usabilidade
RNF02	Diferenciação visual entre via de mão única e mão dupla	0,25	Usabilidade
RNF03	Execução otimizada para grafos grandes (milhares de vértices)	0,25	Desempenho
RNF04	Tempo de resposta inferior a 2 segundos para grafos com ~500 nós	0,25	Desempenho
RNF05	Uso eficiente de memória para evitar sobrecarga em grafos extensos	0,25	Desempenho
RNF06	Interface intuitiva para usuários não técnicos	0,25	Usabilidade
RNF07	Suporte a Windows e Linux	0,25	Portabilidade

ID	Descrição	Pontos	Categoria
RNF08	Código modular e documentado	0,25	Manutenibilidade
Total		2,00	

4. Modelagem UML

4.1 Diagrama de Casos de Uso

A Figura 1 apresenta os casos de uso do sistema, identificando as interações possíveis entre o usuário e o MinRoute.

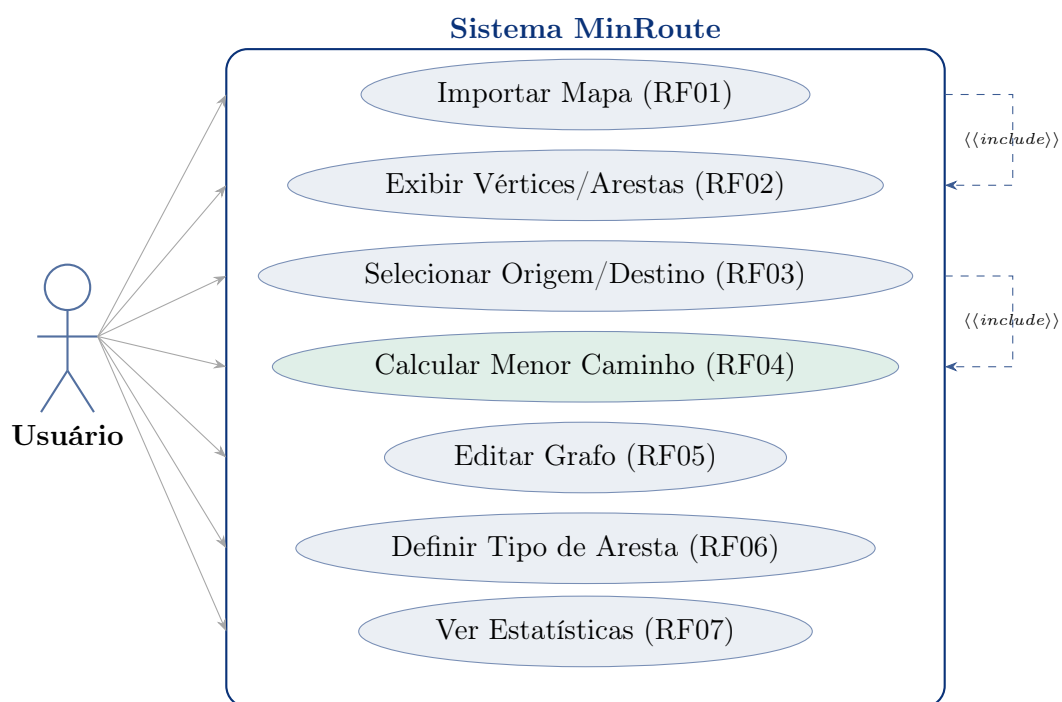


Figure 1: Diagrama de Casos de Uso do MinRoute

4.2 Diagrama de Classes

A Figura 2 apresenta as principais classes do módulo Java e seus relacionamentos, refletindo a implementação real do código-fonte.

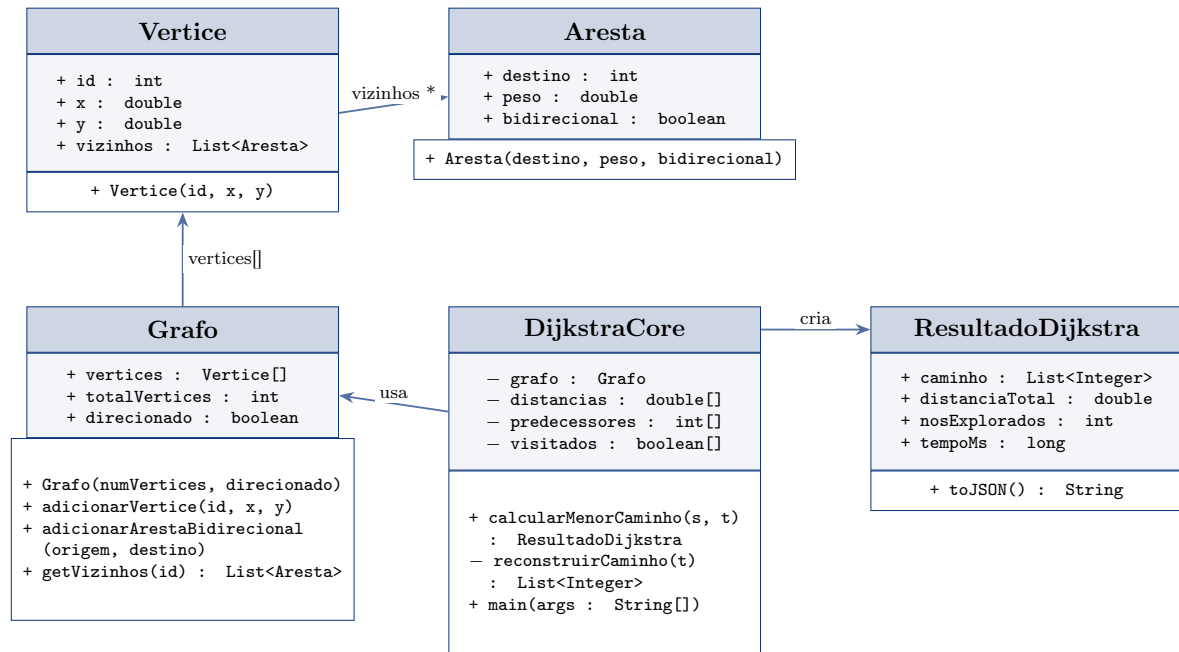


Figure 2: Diagrama de Classes — Módulo Java (Back-end)

4.3 Diagrama de Sequência — Calcular Menor Caminho

A Figura 3 ilustra o fluxo completo de uma requisição de cálculo de rota, desde o clique do usuário até a renderização do resultado.

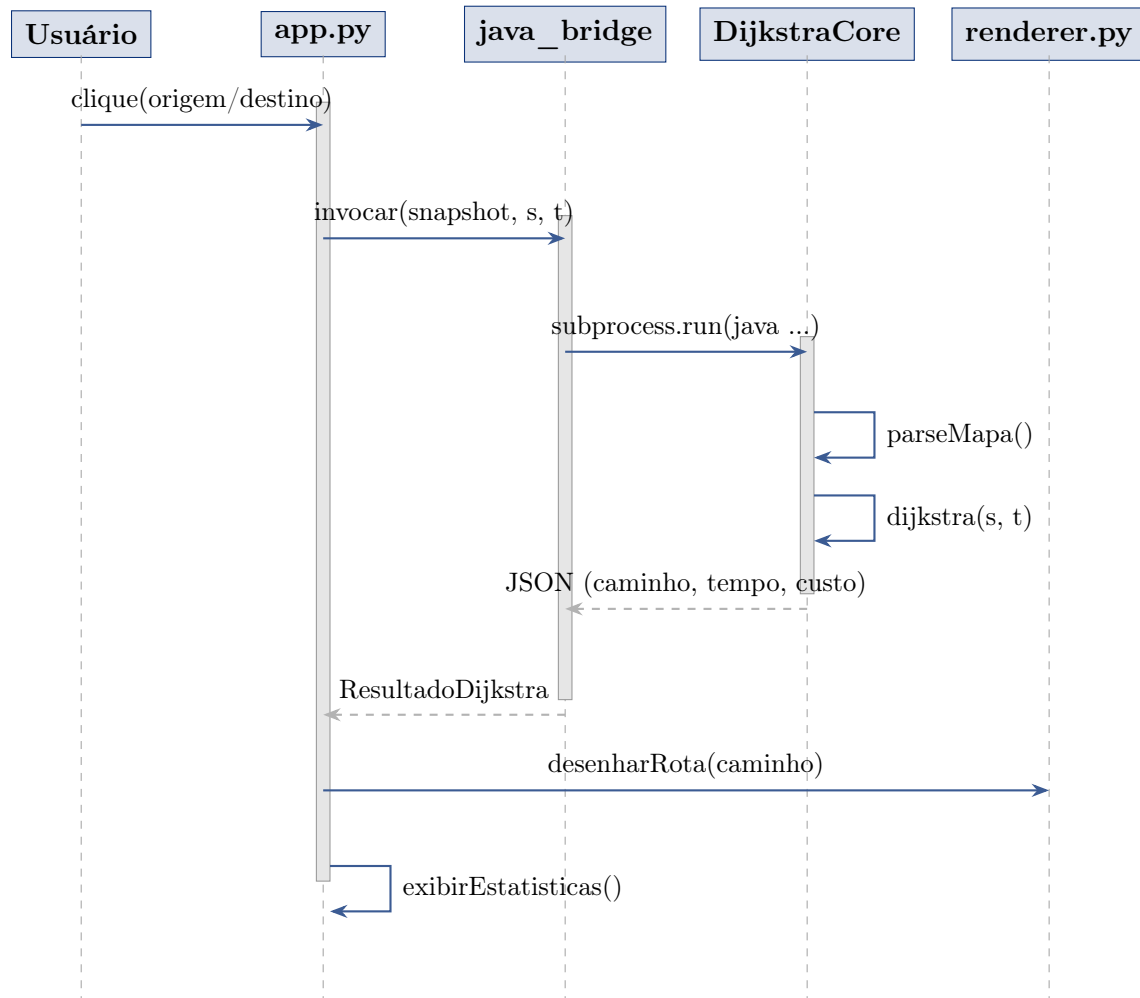


Figure 3: Diagrama de Sequência — Fluxo de Cálculo de Rota

5. Estruturas de Dados

5.1 Lista de Adjacência via Array de Vértices

A representação do grafo utiliza um **array estático de vértices** (`Vertice[] vertices`) em Java, onde cada objeto `Vertice` carrega sua própria `List<Aresta> vizinhos`. A escolha se justifica pelo perfil esparso dos grafos viários reais: cada nó de intersecção conecta-se em média a 2–4 vizinhos, tornando a Matriz de Adjacência ($O(V^2)$ de espaço) ineficiente para redes com milhares de vértices. Além disso, o acesso por índice ao array é $O(1)$, mais rápido que a busca em `HashMap`.

- **Espaço:** $O(V + E)$, proporcional ao número real de conexões
- **Acesso ao vértice por índice:** $O(1)$
- **Inserção de aresta:** $O(1)$ amortizado
- **Iteração sobre vizinhos:** $O(\text{grau}(v))$

Para grafos direcionados (mão única), a aresta é inserida em apenas uma direção. Para grafos não direcionados (mão dupla), o método `adicionarArestaBidirecional` insere em

ambas, calculando o peso automaticamente pela distância Euclidiana entre os vértices.

5.2 Fila de Prioridade — Heap Mínima

O laço de relaxamento do Dijkstra utiliza a `PriorityQueue<NoFila>` nativa do Java, que implementa internamente uma **Heap Mínima binária**.

- **Inserção (offer):** $O(\log V)$
- **Extração do mínimo (poll):** $O(\log V)$
- **Complexidade total do Dijkstra com Heap:** $O(E \log V)$

A classe `NoFila` implementa `Comparable<NoFila>` para ordenar os nós pela distância acumulada.

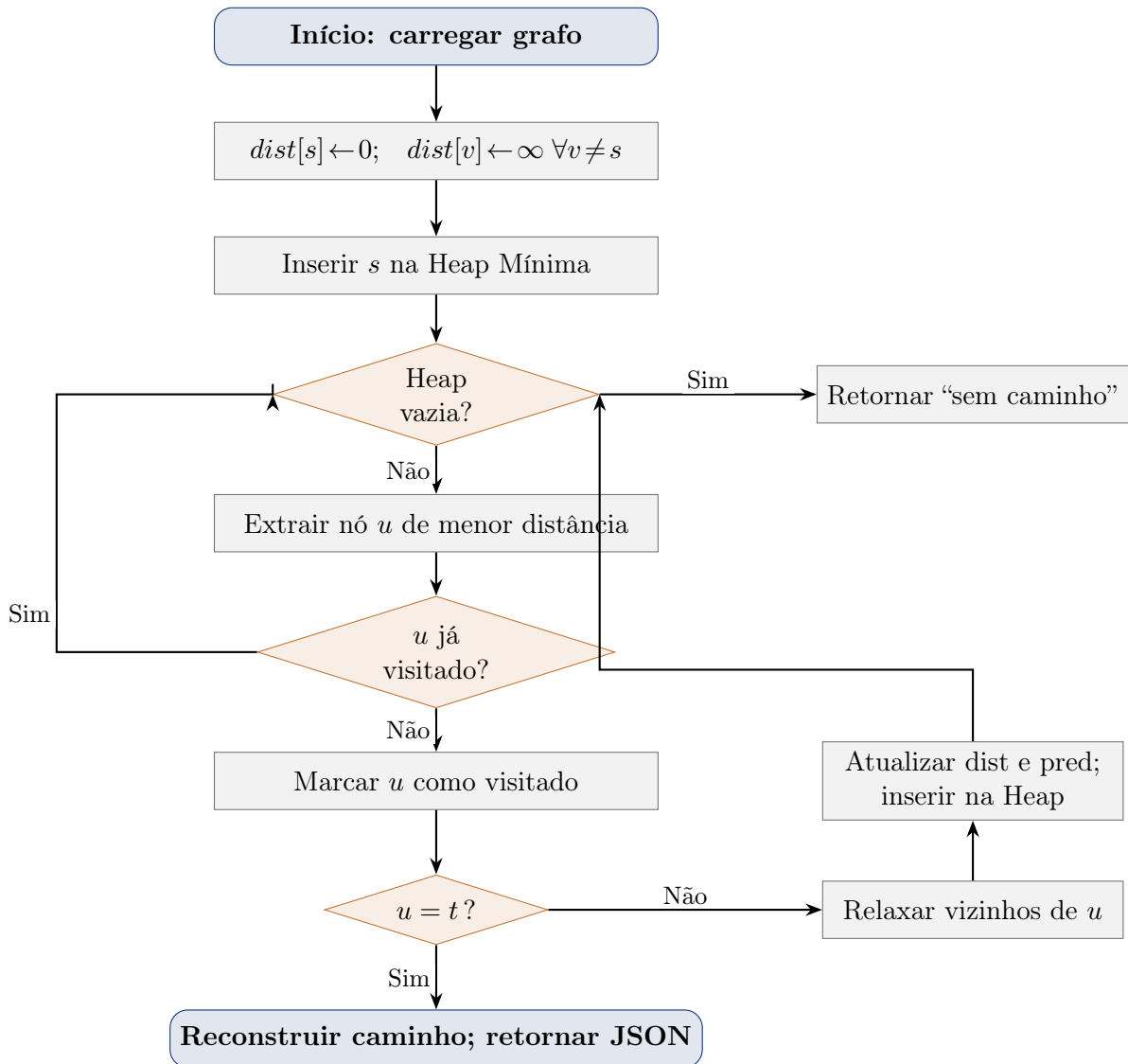
5.3 Vetores de Rastreamento

Três arranjos indexados pela ID dos vértices mantêm o estado do algoritmo:

1. `double[] distancias`: inicializado com `Double.POSITIVE_INFINITY` para todos os vértices, exceto a origem ($d[s] = 0$). Armazena a menor distância conhecida de s a cada nó.
2. `int[] predecessores`: inicializado com `-1`. Quando uma aresta (u, v) é relaxada, registra-se `predecessores[v] = u`. Permite reconstruir o caminho de t até s por *backward tracing*.
3. `boolean[] visitados`: marca os vértices já processados definitivamente. Impede o reprocessamento em grafos com ciclos e garante a terminação do algoritmo.

5.4 Fluxo do Algoritmo de Dijkstra

A Figura 4 apresenta o fluxograma completo da execução do Dijkstra no módulo `DijkstraCore.java`.

Figure 4: Fluxograma do Algoritmo de Dijkstra — `DijkstraCore.java`

6. Arquitetura da Interface Gráfica

6.1 Componentes PyQt6

A interface é construída sobre o framework **PyQt6** seguindo o padrão MVC:

MapGraphicsView (**View**)

Herda de `QGraphicsView`. Gerencia eventos de mouse (pan, zoom via scroll) e converte coordenadas de janela para coordenadas de cena (*scene coordinates*).

renderer.py (**View** — **Renderização**)

Despacha objetos gráficos para a `QGraphicsScene`: `QGraphicsEllipseItem` para vértices e `QGraphicsLineItem` para arestas. Evita redesenho de pixels brutos por frame, mantendo performance em mapas com mais de 10.000 arestas. A rota encontrada é desenhada com espessura e cor diferenciada.

app.py (Controller)

Coordena o estado da aplicação: mantém referências de origem/destino, aciona o `java_bridge`, atualiza a `StatsCard` lateral com as métricas retornadas pelo JSON e trata o reset de seleção.

map_model.py (Model)

Mantém o estado vetorial do grafo em Python. Realiza buscas de nó mais próximo por distância euclidiana, com raio adaptado ao nível de zoom atual. Controla inserção e remoção de vértices e arestas.

6.2 Otimização de Renderização

Para atender ao RNF03 (grandes grafos), são empregadas duas estratégias:

1. **Level of Detail (LOD):** O `QGraphicsScene` utiliza *bounding rects* para ignorar o processo de pintura de componentes fora do campo de visão (*viewport*), reduzindo ciclos de CPU.
2. **Reutilização de objetos:** Itens gráficos são atualizados, não recriados, a cada mudança de estado.

6.3 Interface de Comunicação IPC (JSON)

O protocolo de comunicação entre Python e Java é baseado em JSON via `STDOUT`. O `java_bridge.py` invoca o executável Java com três argumentos: `[arquivo_mapa, id_origem, id_destino]`.

Em caso de sucesso, o `DijkstraCore` retorna:

```
1 {
2   "tempo_ms": 32,
3   "nos_explorados": 412,
4   "distancia_total": 5420.55,
5   "caminho": [3, 14, 85, 204, 39]
6 }
```

Listing 2: Resposta JSON do `DijkstraCore`

Em caso de caminho inexistente (grafo desconexo), retorna:

```
1 {
2   "tempo_ms": 18,
3   "nos_explorados": 210,
4   "distancia_total": -1,
5   "caminho": []
6 }
```

Listing 3: Resposta JSON quando não há caminho

7. Tratamento de Exceções e Casos Especiais

Grafo desconexo

O `Dijkstra` termina com `distancias[t] == Double.POSITIVE_INFINITY`. O back-

end serializa "distancia_total": -1 e "caminho": []. O front-end exibe um diálogo de aviso ao usuário.

Clique em área vazia

O `map_model.py` verifica a distância euclidiana do ponto clicado a todos os vértices visíveis. Se nenhum estiver dentro do raio de tolerância (adaptado ao zoom), o evento é ignorado no modo de navegação. No modo de edição, cria um novo vértice na posição.

Arquivo de mapa inválido

Os parsers Java lançam `IllegalArgumentException` com mensagem descritiva. O `java_bridge.py` captura o código de retorno do subprocesso e exibe o erro no painel lateral.

Peso negativo

O Dijkstra é definido apenas para pesos não negativos. O sistema valida os pesos durante a importação e rejeita arestas com peso < 0 .

8. Guia de Instalação e Execução

8.1 Pré-Requisitos

- **Java:** JDK 11 ou superior (`java -version` para verificar)
- **Python:** 3.9 ou superior (`python -version` para verificar)
- **Sistema operacional:** Windows 10+ ou Linux (Ubuntu 20.04+)

8.2 Instalação das Dependências Python

```
1 pip install PyQt6 python-dotenv
```

8.3 Compilação do Módulo Java

A partir da raiz do repositório, compilar todos os arquivos `.java` e direcionar os `.class` para `build/classes/`:

```
1 javac -d build/classes src/core/*.java
```

8.4 Execução

```
1 python src/ui/app.py
```

Após a janela abrir, utilize o botão **Importar Mapa** para carregar um arquivo da pasta `data/`. Em seguida, clique em dois pontos do canvas para definir origem e destino e pressione **Traçar Menor Caminho** (ou `Ctrl+C`).

8.5 Instalador Autônomo

Para distribuição sem ambiente de desenvolvimento, o projeto disponibiliza o arquivo `Setup.exe`, gerado via *PyInstaller*, que empacota o interpretador Python, as dependências PyQt6 e os binários Java compilados. Não requer instalação prévia de Python ou JDK.

9. Referências

- [1] CORMEN, T. H. et al. *Introduction to Algorithms*. 4. ed. MIT Press, 2022. Cap. 22–24 (Grafos e Dijkstra).
- [2] SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Addison-Wesley, 2011. Cap. 4.4 (Shortest Paths).
- [3] OPENSTREETMAP CONTRIBUTORS. *OpenStreetMap*. Disponível em: <https://www.openstreetmap.org>. Acesso em: mai. 2026. (Fonte dos mapas .osm utilizados.)
- [4] Qt GROUP. *Qt for Python — PyQt6 Documentation*. Disponível em: <https://doc.qt.io/qtforpython/>. Acesso em: mai. 2026.
- [5] ORACLE. *Java SE 11 Documentation — PriorityQueue*. Disponível em: <https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/PriorityQueue.html>. Acesso em: mai. 2026.